

Documentación - Análisis Semántico y Generación Código

Intermedio

Estudiantes

Geovanni González Aguilar - 2022163324

Owen Torres Porras - 2023302034

Escuela de Ingeniería en Computación

Curso IC5701 - Compiladores e Intérpretes

Profesor

Allan Rodríguez Dávila

Grupo

60

II Semestre 2026

Tabla de Contenidos

Tabla de Contenidos.....	2
Manual de Usuario.....	3
Pruebas de Funcionalidad.....	7
Descripción del Problema.....	10
Diseño del Programa.....	11
Librerías y Herramientas Usadas.....	12
JFlex.....	12
CUP.....	12
java-cup-runtime.....	13
Análisis de Resultados.....	13
Justificación de la Toma de Decisiones.....	14
Bitácora.....	15

Manual de Usuario

El compilador se ejecuta desde consola mediante un archivo JAR generado por Maven. El usuario proporciona un archivo fuente y, opcionalmente, un directorio de salida donde se almacenan los reportes y el código intermedio.

Linux

1. Instalacion de Maven

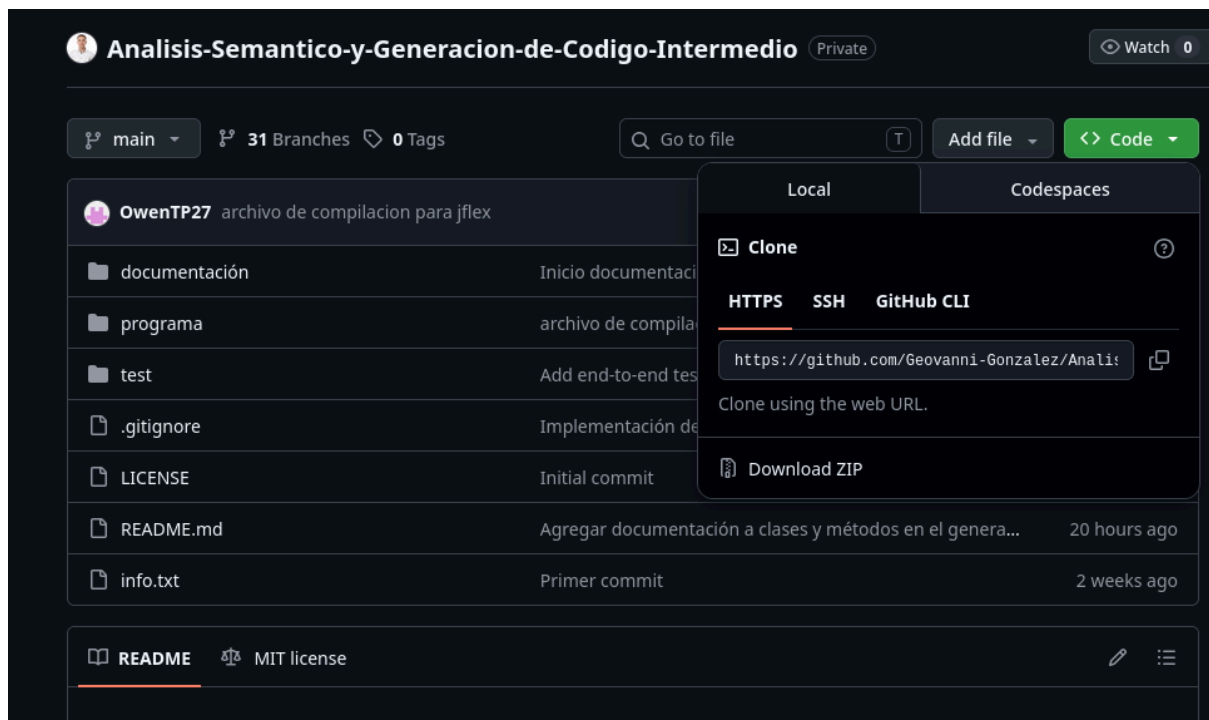
Para instalar maven en linux se puede usar el siguiente comando:

Arch linux: `sudo pacman -S maven`

Ubuntu: `sudo apt install maven`

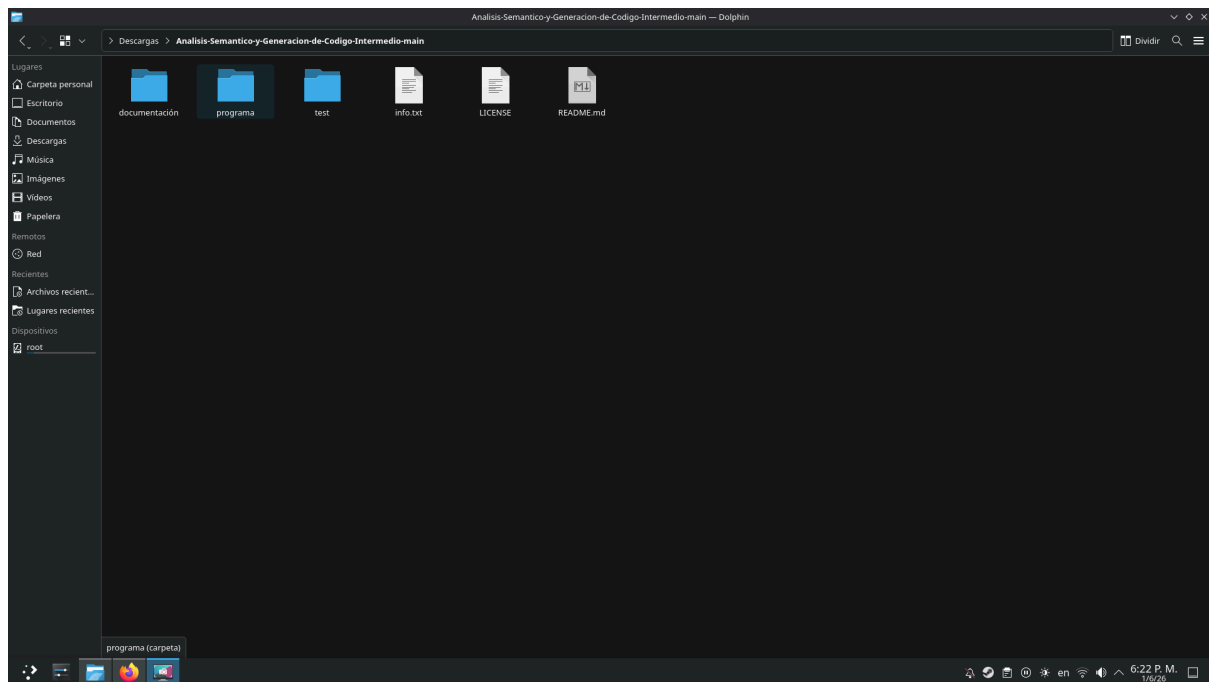
2. Descargar proyecto

Para ejecutar el programa es necesario descargar el zip del repositorio <https://github.com/Geovanni-Gonzalez/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio.git>



3. Iniciar la aplicación

Una vez descargado y descomprimido entramos a programa



dentro de la carpeta programa abriremos una terminal y ejecutaremos el comando: mvn clean

```

programa : fish — Konsole

-----
otp@cachyos
-----
OS: CachyOS x86_64
Kernel: Linux 7.0.9-1-cachyos
Uptime: 7 hours, 34 mins
Packages: 1451 (pacman)
Shell: fish 4.7.1
Display (VG27AQL3A): 1920x1080 in 27", 120 Hz [External]
DE: KDE Plasma 6.6.5
WM: KWin (Wayland)
WM Theme: Breeze
Theme: Breeze (Dark) [Qt], Breeze-Dark [GTK2], Breeze [GTK3]
Icons: breeze-dark [Qt], breeze-dark [GTK2/3/4]
Font: Noto Sans (10pt) [Qt], Noto Sans (10pt) [GTK2/3/4]
Cursor: breeze (24px)
Terminal: konsole 26.4.1
CPU: AMD Ryzen 5 3600XT (12) @ 4.63 GHz
GPU: NVIDIA GeForce GTX 1050 [Discrete]
Memory: 6.31 GiB / 15.53 GiB (41%)
Swap: 1.16 GiB / 15.53 GiB (7%)
Disk (/): 91.88 GiB / 927.42 GiB (10%) - btrfs
Local IP (wlan0): 192.168.1.11/24
Locale: es_MX.UTF-8

~/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa
> mvn clean
[INFO] Scanning for projects...
[INFO] -----< cr.ac.itcr:proyecto-compiladores >-----
[INFO] Building proyecto-compiladores 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- clean:3.2.0:clean (default-clean) @ proyecto-compiladores ---
[INFO] BUILD SUCCESS
[INFO] Total time: 0.154 s
[INFO] Finished at: 2026-06-01T18:28:00-06:00
[INFO] -----
~/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa
>

```

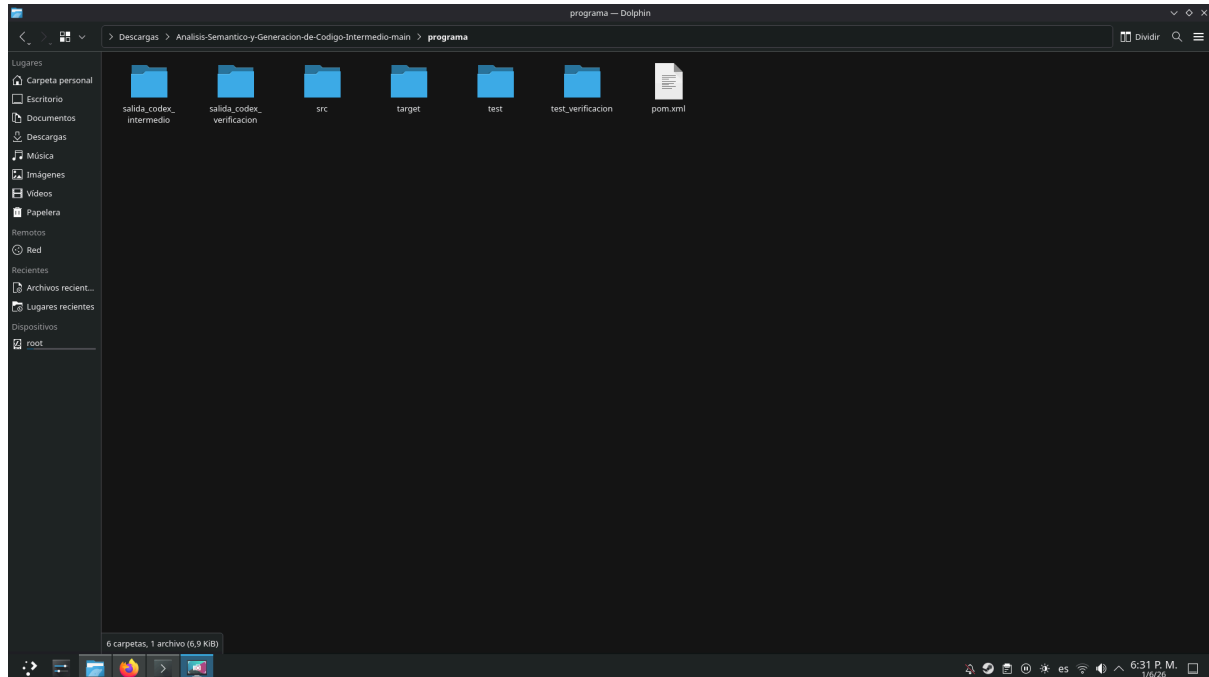
Y luego: mvn package

```

~/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa
> mvn package
[INFO] Scanning for projects...
[INFO] -----< cr.ac.itcr:proyecto-compiladores >-----
[INFO] Building proyecto-compiladores 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- ant:run:3.1.0:run (generate-parser) @ proyecto-compiladores ---
[INFO] Executing tasks
[INFO] [mkdir] Created dir: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/target/generated-sources/cup
[INFO] [java] ----- CUP v0.11a beta 20060608 Parser Generation Summary -----
[INFO] [java] 0 errors and 0 warnings
[INFO] [java] 58 terminals, 49 non-terminals, and 200 productions declared,
[INFO] [java] producing 411 unique parse states.
[INFO] [java] 0 terminals declared but not used.
[INFO] [java] 0 non-terminals declared but not used.
[INFO] [java] 0 productions never reduced.
[INFO] [java] 0 conflicts detected (0 expected).
[INFO] [java] Code written to "Parser.java", and "sym.java".
[INFO] [java] ----- (v0.11a beta 20060608)
[INFO] Executed tasks
[INFO] --- jflex:1.0.1:generate (generate-lexer) @ proyecto-compiladores ---
[INFO] generated /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/target/generated-sources/jflex/lexico/MiLexer.java
[INFO] --- build-helper:3.4.0:add-source (add-generated-sources) @ proyecto-compiladores ---
[INFO] Source directory: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/target/generated-sources/cup added.
[INFO] Source directory: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/target/generated-sources/jflex added.
[INFO] --- resources:3.4.0:resources (default-resources) @ proyecto-compiladores ---
[INFO] skip non existing resourceDirectory /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/src/main/resources
[INFO]

```

esto construirá una carpeta target en la carpeta programa



dentro de la misma terminal para ejecutar el programa se necesita entrar a la carpeta target para usar el archivo proyecto-compiladores-1.0-snapshot.jar y una dirección de memoria a un archivo de prueba dando como resultado un comando así:

`java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar direccion/prueba`

por ejemplo:

`java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar`

`test_verificacion/10_funciones.chip`

```
~/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa
> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar test_verificacion/10_funciones.chip
El archivo fuente puede ser generado por la gramatica.
Reporte de tokens: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tokens_report.txt
Tabla de simbolos: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tabla_simbolos.txt
Reporte de errores: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/errores_report.txt
Resultado sintactico: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/resultado_sintactico.txt
```

Una vez ejecutado dentro de la carpeta programa se construirá una carpeta salida que contendrá los archivos de reportes.

Windows

Requisitos:

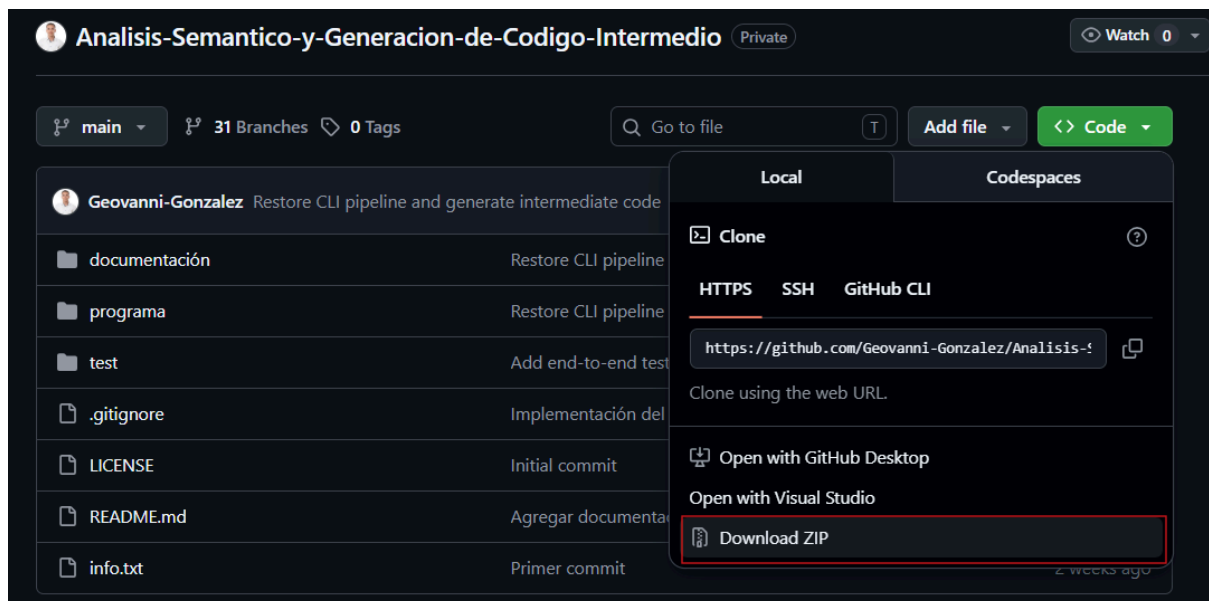
- JDK 8 o superior instalado

1. Instalar Maven

Chocolatey: `choco install maven -y`

2. Descargar .zip del repositorio

Ingresa al [link](#) del repositorio y descarga el archivo .zip



3. Compilar

Una vez descargado y descomprimido el proyecto de forma local, abra una consola de

PowerShell y ubíquese dentro de la carpeta principal del programa

```
cd .\Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main\programa
```

Posteriormente, compile el proyecto utilizando Maven. Este proceso limpiará los archivos generados anteriormente, compilará el código fuente y creará el archivo ejecutable **.jar** dentro de la carpeta **target**.

```
mvn clean package
```

Una vez finalizada correctamente la compilación, ejecute el programa mediante el archivo **.jar** generado. Para ello, utilice el siguiente comando:

```
java -jar target\proyecto-compiladores-1.0-SNAPSHOT.jar <archivo_entrada>  
<ruta_salida>
```

Donde:

<archivo_entrada>

corresponde a la ruta del archivo fuente que será analizado por el programa.

<ruta_salida>

corresponde a la ruta o nombre base donde se generarán los archivos de salida producidos por el sistema.

Por ejemplo, si se desea analizar un archivo ubicado en la carpeta test, el comando podría tener una estructura similar a la siguiente:

```
java -jar target\proyecto-compiladores-1.0-SNAPSHOT.jar  
..\test\nombre_archivo.lang ..\test\nombre_salida
```

Pruebas de Funcionalidad

1. Prueba archivo asignación de tipado fuerte

```
empty ~ __main__ <| |> |:
```

```
int ~ x <- 1!
```

```
float ~ y <- 1.5!
```

```
x <- 2!
```

```
x <- 2.5!
```

```
y <- 3.5!
```

```
noDeclarada <- 1!
```

```
:|
```

Este es el resultado de compilación:

```
~/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa
$ java -jar target/proyecto-compiladoras-1.0-SNAPSHOT-jar-test-verificacion/21_asignaciones_tipado_fuerte.chip
Error semantico [línea 5, col 1]: asignacion incompatible para 'x': se esperaba tipo int y se obtuvo tipo float
Error semantico [línea 7, col 1]: variable 'noDeclarada' no declarada
Archivo analizado: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/test_verificacion/21_asignaciones_tipado_fuerte.chip
El archivo fuente NO puede ser generado por la gramatica.
Reporte de tokens: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tokens_report.txt
Tabla de simbolos: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tabla_simbolos.txt
Reporte de errores: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/errores_report.txt
Resultado sintactico: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/resultado_sintactico.txt
```

Hay un error debido se le intenta asignar a un entero un flotante y se intenta asignar un valor a una variable que no ha sido declarada

2. Prueba archivo if else

```
empty ~ __main__ <| |>
```

```
:|
```

```
int ~ x <- 10 !
```

```
if <|greater_t<|x,5|>|>
```

```
:|
```

```
cout <|x|> !
```

```
:|
```

```

else

|:

    cout <|"menor"|> !

|:

|:

```

```

~/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa
> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar test_verificacion/10_funciones.chip
Archivo analizado: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/test_verificacion/10_funciones.chip
El archivo fuente puede ser generado por la gramatica.
Reporte de tokens: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tokens_report.txt
Tabla de simbolos: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tabla_simbolos.txt
Reporte de errores: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/errores_report.txt
Resultado sintactico: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/resultado_sintactico.txt

```

Se compila sin errores y da como resultado este código intermedio:

```

// Codigo intermedio
// Fecha: 2026-06-01
// Archivo fuente: 10_funciones.chip
// Integrantes: Geovanni Gonzalez

>> begin_function sumar
>>   _t0 = a + b
>>   return _t0
>> end_function sumar
>> begin_function __main__
>>   param 2
>>   param 3
>>   _t1 = call sumar, 2
>>   resultado = _t1
>>   print resultado
>> end_function __main__

```

3. Prueba archivo do while

```

empty ~ __main__<|>

|:

    int ~ x <- 0 !

do

|:

    x <- x + 1 !

```

:

while <|less_t<|x,10|>|> !

:

El resultado es el siguiente:

```
> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar test_verificacion/08_do_while.chip
Archivo analizado: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/test_verificacion/08_do_while.chip
El archivo fuente puede ser generado por la gramatica.
Reporte de tokens: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tokens_report.txt
Tabla de simbolos: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/tabla_simbolos.txt
Reporte de errores: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/errores_report.txt
Resultado sintactico: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/resultado_sintactico.txt
Codigo intermedio: /home/otp/Descargas/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio-main/programa/salida/08_do_while.ic
```

se compila sin errores y da este código intermedio como resultado

```
// Codigo intermedio
// Fecha: 2026-06-01
// Archivo fuente: 08_do_while.chip
// Integrantes: Giovanni Gonzalez

> begin_function __main__
> x = 0
_L0:
> _t0 = x + 1
> x = _t0
> _t1 = x < 10
> if_false _t1 goto _L1
> goto _L0
_L1:
> end_function __main__
```

Descripción del Problema

El proyecto responde a la necesidad de construir un compilador para un lenguaje imperativo ligero orientado a la configuración de chips y al desarrollo de sistemas empujados. El compilador debe procesar programas formados por funciones, declaraciones, expresiones, estructuras de control, entrada/salida y retornos, validando que el código cumpla con las reglas léxicas, sintácticas y semánticas definidas.

El problema principal consiste en pasar de una secuencia de caracteres escrita por el usuario a una representación intermedia confiable. Para lograrlo, el sistema debe clasificar tokens,

construir una estructura sintáctica válida, verificar tipos y alcances, reportar errores recuperables y evitar que programas inválidos generen código intermedio.

Además, se requiere que exista exactamente un método `__main__`, que las llamadas a funciones respeten cantidad y tipo de argumentos, que las condiciones sean booleanas y que los retornos coincidan con el tipo declarado de cada función.

Diseño del Programa

Arquitectura general

El diseño se separa por capas para mantener responsabilidades claras: análisis léxico, análisis sintáctico, construcción del AST, análisis semántico, generación de código intermedio y escritura de reportes.

Analizador léxico

El analizador léxico se genera con JFlex a partir de `lexico.flex`. Su función es recorrer el archivo fuente, reconocer lexemas y producir tokens con información de línea, columna y categoría.

Analizador sintáctico

El parser se genera con CUP a partir de `sintactico.cup`. Valida que la secuencia de tokens respete la gramática del lenguaje y construye nodos del AST mediante acciones semánticas.

Análisis semántico

El paquete semántico administra la tabla de símbolos, los alcances, las firmas de funciones, el tipo de expresiones y las reglas de compatibilidad. Los errores semánticos se acumulan para producir reportes claros y evitar cascadas innecesarias.

Código intermedio

El paquete intermedio traduce el AST aceptado a instrucciones de tres direcciones. Usa temporales como `_t0`, etiquetas como `_L0`, instrucciones `param/call` para funciones, saltos condicionales `if_false` y `print` para salida observable.

Reportes

El paquete reporte centraliza la escritura de tokens, tabla de símbolos, errores, resultado sintáctico y archivos `.ic`. Esto evita duplicar lógica de salida en `Main.java` o en las pruebas.

Librerías y Herramientas Usadas

Herramienta/Librería	Versión	Función
JFlex	1.9.1	Generador del analizador léxico desde <code>lexico.flex</code>
CUP (java-cup)	11a	Generador del parser LALR(1) desde <code>sintactico.cup</code>
java-cup-runtime	11b	Runtime con clases base del parser (<code>lr_parser</code> , <code>Symbol</code> , <code>Scanner</code>)
Java JDK	8+	Compilación y ejecución del sistema completo

JFlex

JFlex es un generador de analizadores léxicos para Java. A partir de un archivo .flex con expresiones regulares y acciones Java, genera automáticamente una clase Java que implementa el scanner.

CUP

CUP es un generador de parsers para Java. A partir de una gramática en formato BNF con acciones semánticas, genera el parser y la clase de símbolos.

java-cup-runtime

Contiene las clases base que el parser generado por CUP necesita en tiempo de ejecución: Ir_parser (clase padre del parser), Symbol (representación de tokens y no terminales) y Scanner (interfaz que debe implementar el lexer). Debe incluirse en el classpath tanto al compilar como al ejecutar el proyecto.

Análisis de Resultados

Durante las pruebas se identificaron dos defectos de diseño relevantes. El primero fue que buscar insertaba un símbolo de error persistente en la tabla cuando una variable no era encontrada, lo que provocaba que una declaración posterior del mismo identificador reportara falsamente una redeclaración en lugar del error real; la corrección consistió en eliminar insertarSimboloError del flujo de buscar, retornando el símbolo de error de forma transitoria sin persistirlo. El segundo defecto fue de naturaleza gramatical: el operando derecho de los operadores lógicos # (OR) y @ (AND) estaba restringido a expresionRelTernaria, impidiendo

que el operador de negación \$ (NOT) apareciera como operando directo de una expresión disyuntiva o conjuntiva; esto se corrigió elevando el operando a `expresionLog` y declarando precedencia explícita en CUP con `precedence left OR_LOG`, `precedence left AND_LOG` y `precedence right NOT_LOG`, resolviendo los seis conflictos `shift/reduce` generados y manteniendo la semántica estándar de los operadores lógicos.

Justificación de la Toma de Decisiones

La decisión de separar el analizador semántico en dos clases con responsabilidades distintas responde al principio de separación de incumbencias: `AnalizadorSemantico` concentra la lógica de validación y sirve como punto de contacto con el parser, mientras que `TablaDeSimbolos` encapsula exclusivamente la gestión de alcances y el reporte de errores, lo que facilita el mantenimiento y la extensión independiente de cada componente. El uso de una pila de mapas para los alcances fue la elección natural para un lenguaje con bloques anidados, ya que permite abrir y cerrar alcances en tiempo constante y resolver identificadores recorriendo la pila de arriba hacia abajo, respetando el sombreado de variables. La propagación de `TipoDato.ERROR` en la evaluación recursiva de expresiones fue una decisión deliberada para evitar errores en cascada: una vez que un subárbol falla, los nodos padre lo absorben sin generar reportes adicionales que confundirían al programador con falsos positivos. La estrategia de registro de funciones en dos fases, inserción con firma vacía seguida de actualización con los tipos reales, fue necesaria porque la gramática procesa

los parámetros después de haber abierto el alcance de la función, de modo que separar ambas operaciones permite mantener el orden natural del parser sin reestructurar la gramática. Finalmente, la corrección de los conflictos shift/reduce mediante declaraciones de precedencia en lugar de refactorizar la gramática en múltiples no terminales fue una decisión pragmática: introducir niveles adicionales como `expresionAnd` y `expresionOr` habría resuelto la ambigüedad de forma más explícita, pero habría incrementado la cantidad de producciones y dificultado la lectura de la gramática, mientras que las directivas `precedence` de CUP logran el mismo efecto con menor impacto estructural y de forma declarativa.

Bitácora

Enlace al github:

<https://github.com/Geovanni-Gonzalez/Analisis-Semantico-y-Generacion-de-Codigo-Intermedio.git>